Relative vs. Absolute Positional Encoding Predicts Position Generalization at Fixed Length in Micro-Transformers

John Lee ’27 · Advisor: Prof. John MacCormick
Deep Network Understanding Lab, Department of Mathematics and Computer Science, Dickinson College

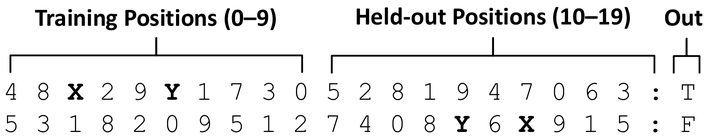
Introduction

A transformer model needs a **positional encoding** to see where each token is located. There are two main families. **Absolute** gives every position its own value (learned, sinusoidal). **Relative** uses only how tokens relate to each other (RoPE, T5, NoPE).
In our study, we compare all five. The symbols that decide the answer appear in the left half during training and the right half at test, and the length never changes.

Does the positional encoding decide whether the model generalizes to the right half, where it never saw those symbols?

Task

Vocabulary: ‘0’ - ‘9’, ‘X’, ‘Y’, ‘:’, ‘T’, ‘F’
Every input is 20 characters. **Output ‘T’ if ‘X’ comes before ‘Y’, else ‘F’.**
Model sees everything up to ‘:’ and must produce the final token.



Setup

Model	0.8M Parameters: 4 layers, 4 heads, width 128, causal. Trained from scratch on a laptop CPU.
Data (per seed)	50,000 (training), 5,000 (validation), 2,000 (testing)
Runs	5 positional encoding methods × 10 seeds = 50 runs

Positional Encoding Methods

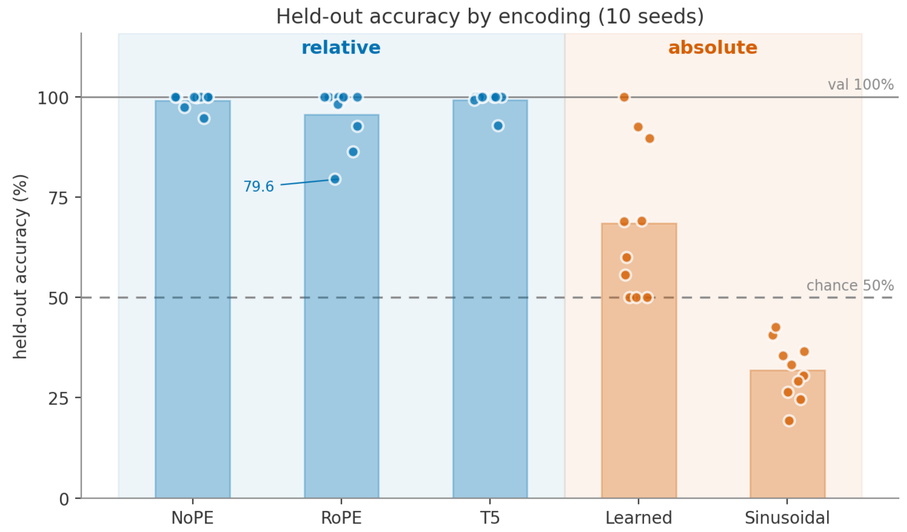
Absolute adds a position vector to the embedding: `token + p_i`
Relative sees only the gap, inside attention: `f( j − i )`

METHODS	FAMILY	HOW IT ADDS POSITION
NoPE	Relative	Nothing; order comes from the causal mask
RoPE	Relative	Rotates q and k inside attention
T5	Relative	Learned bias by distance, inside attention
Learned	Absolute	A trainable vector added to each token
Sinusoidal	Absolute	A fixed vector added to each token

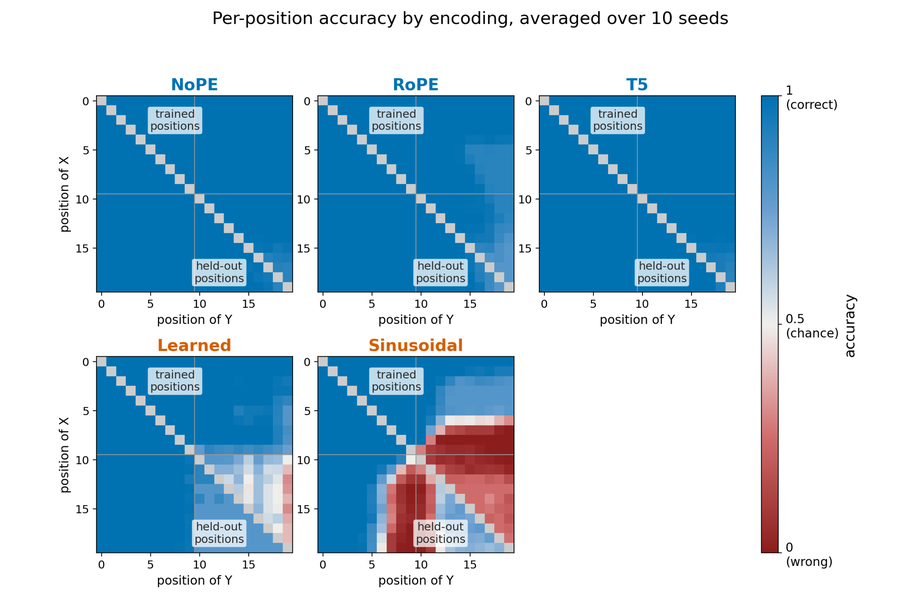
Key Findings

Only the **relative encodings** generalize to held-out positions.
All five methods reach 100% accuracy on the trained positions.
The **absolute ones (learned, sinusoidal) fall apart on held-out positions**.
And sinusoidal does worse than guessing.

Held-out accuracy



Per-position accuracy



Why absolute methods fail

With an **absolute encoding**, every position carries its own signal, so the model can memorize an answer for each pair of positions it saw instead of learning the rule.
Learned trains a vector for all twenty positions, but the right-half vectors are only ever trained by filler digits, never by an X-and-Y decision. Whether they work at test depends on the seed.
Sinusoidal's position vectors come from a fixed formula, so they are identical in every run, and every run lands below chance.
Neither failure is about positions the model never saw. It saw all twenty, in every example.

Conclusion

On this task, **the family of the positional encoding predicts whether the model generalizes at all.** All three relative encodings generalize to held-out positions. Neither absolute one does reliably.
How they fail is a different matter, and the two absolute encodings do not fail alike. **Sinusoidal** is below chance in every run. Learned is sometimes right, but which runs work depends on the seed.
RoPE ranks poorly when models are trained on short inputs and tested on longer ones (Kazemnejad et al., 2023), yet it does well here. Position and length may be different problems.

Limitations & next steps

One task, one split, one length. Whether the relative/absolute divide holds for other position-dependent tasks is open.
Both halves cover the same range of gaps between X and Y, so a relative encoding sees nothing new at test. The split is not a hard test for that family. What it does expose is how the absolute ones break.

Next: shrink the model to the smallest one that still solves the task, add scratchpad tokens, and look inside the trained models to see what they learned.

References

1. Vaswani et al. (2017). Attention is all you need. NeurIPS.
2. Radford et al. (2018). Improving language understanding by generative pre-training. OpenAI technical report.
3. Raffel et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR 21(140).
4. Su et al. (2021). RoFormer: enhanced transformer with rotary position embedding. arXiv:2104.09864.
5. Kazemnejad et al. (2023). The impact of positional encoding on length generalization in transformers. NeurIPS.

Code & Data: github.com/JinLeeGG/comp560-JohnLee